COLLEGE OF
INNOVATION & TECHNOLOGY

Algorithm and Complexity Analysis

CSC-379-01 / CSC-575-01

PROJECT- Fall 2025

Deadline: 11/24/2025 @ 23:59

[Total Mark for this Assessment is 100]

Student Details:

Name: Bradley Moore **CRN: 35737785**	ID: CSC-580

Submission Instructions:

- This assignment must be submitted on Moodle **pdf format only**).
- The page number should appear on each page.
- All the text and diagrams must be electronically produced.
- Email submission will not be accepted.
- You are advised to make your work clear and well-presented; marks may be reduced for poor presentation. This includes filling your information on the cover page.
- You MUST show all your work, and text <u>must not</u> be converted into an image, unless specified otherwise by the question.
- The project proposal should contain the following sections:

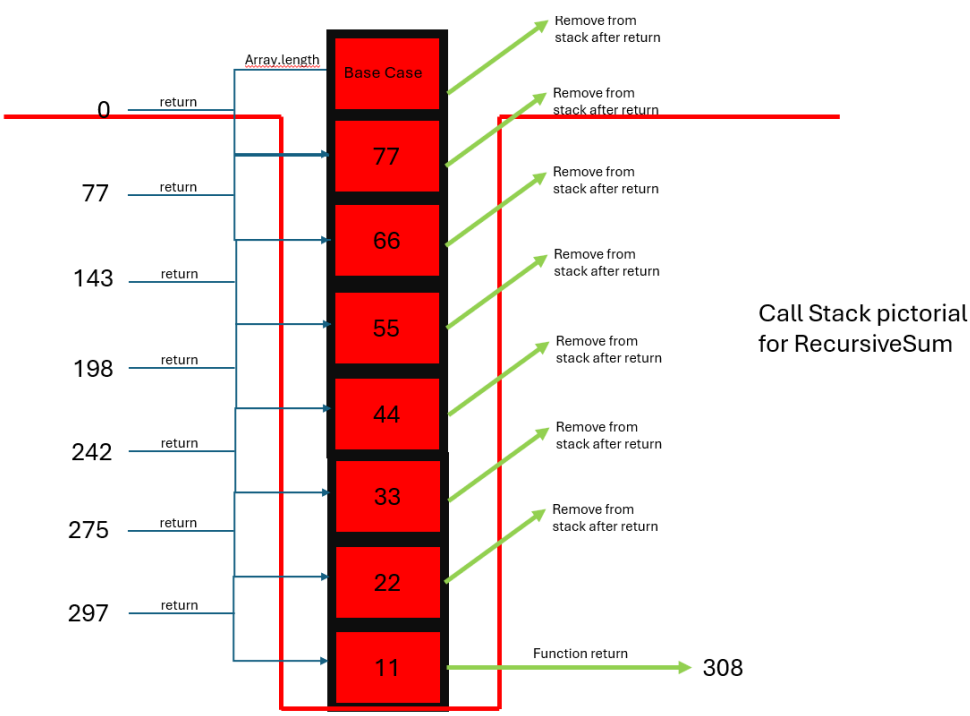
Question one

(25 Marks)

Suppose you have the following array: {11, 22, 33, 44, 55, 66, 77}. **Write a java program to find the sum of elements of the array recursively.**

```
public class RecursiveSum{

   public static void main(String[] args) {


      int[] numbers = {11,22,33,44,55,66,77}; // Define array


      int sum = recursiveSum(numbers, 0); // Call Recursive Function starting at the beginning of the array


      System.out.println("The total of the array is: " + sum);//Print output


   }

   /**

    * Function Name: recursiveSum

    *

    * @param array

    * @param index

    * @return

    *

    * Inside the function:
```

```
    * 1. establishes base case for recursion, returning 0 when all indeces have been gone through

    * 2. recursive call to sum all integers. Moves to next index in recursion

    *

    */

   public static int recursiveSum(int[]array, int index){

      if (index == array.length){

         return 0;

      }

      return array[index] + recursiveSum(array, index+1);

   }

}
```



Question Two

(*25 Marks*)

Use the following method to generate an array (arr1) of 5000 randomly generated elements.

```
public static int[] RandomArrayGenerator(int n)
  {
    int[] Array = new int[n];
    Random r = new Random();
    for(int i=0;i<n;i++)
    {
      Array[i] = r.nextInt(1000);
    }
    return Array;
  }
```

[Note that by using nextInt(100) you will generate an integer between 0 and 100.]

a. Use insertion sort to sort arr1.

b. Use RandomArrayGenerator again to create arr2 containing 5000 randomly generated element.

c. Use the static sort methods of Java's Arrays class to sort arr2.

d. Time each of the methods so that you can make a comparison of the efficiency. You can create longs and store the system time to know when the method starts and ends. You can then take the difference run time in milliseconds.

long startTime = System.currentTimeMillis();

//call sort method here

long endTime = System.currentTimeMillis();

Solution

```
//import random module
import java.util.Random;

//import Arrays Method
import java.util.Arrays;

public class RandomArrays{
   public static void main(String[] args) {

      //Generate array1 of 5000 random elements
      int[] arr1 = RandomArrayGenerator(5000);

      //Start timer
      long startTime1 = System.currentTimeMillis();

      //call insertion sort
      insertionSort(arr1);

      //End Timer
      long endTime1 = System.currentTimeMillis();

      //save elapsed time1
      long elapsedTime1 = endTime1 - startTime1;

      //Generate array2 of 5000 random elements
      int[] arr2 = RandomArrayGenerator(5000);

      //start Timer
      long startTime2 = System.currentTimeMillis();

      //call primitive (double-pivot) array sorting
      Arrays.sort(arr2);
```

```
    //save run time
    long endTime2 = System.currentTimeMillis();

    //save elapsed time2
    long elapsedTime2 = endTime2 - startTime2;

    //Generate array3 of 5000 random elements
    int[] arr3 = RandomArrayGenerator(5000);

    //start Timer
    long startTime3 = System.currentTimeMillis();

    //call parralel array sorting
    Arrays.parallelSort(arr3);

    //save run time
    long endTime3 = System.currentTimeMillis();

    //save elapsed time2
    long elapsedTime3 = endTime3 - startTime3;

    //print message
    System.out.println("Insertion Sort time = " + elapsedTime1+"ms");
    System.out.println("Primitive Sort time = "+ elapsedTime2+"ms");
    System.out.println("Parallel Sort time = "+ elapsedTime3+"ms");
  }
  public static int[] RandomArrayGenerator(int n)
{
  int[] Array = new int[n];
  Random r = new Random();
  for(int i=0;i<n;i++)
  {
```

```
    Array[i] = r.nextInt(1000);
  }
  return Array;
}

/**
 * Function Name: Insertion Sort
 *
 * @param array
 *
 * Inside The Function:
 * 1. Loops through array comparing each value to the value at the previous index
 * 2. inserts current value into proper index when it meets a value at a previosu index
that is less
 *
 */
public static void insertionSort(int[] array){
  //loop through the array starting at the second index
  for (int i=1; i<array.length; i++){
      //temp variable to store value at index
      int temp = array[i];
      //variable to access value store at indeces before i starting at the index i-1
      int j = i-1;

      //start while loop to move value towards front of array until j gets to beginning and
the value is less than the previous value
      while (j> -1 && temp < array[j]){
         // set previous = to current
        array[j+1] = array[j];
        //set current = previous
        array[j] = temp;
        //increment j
        j--;
```

```
        }
    }
  }

}
```

Output

```
PS C:\Users\YZRD25\Documents\GitHub\ai-masters-workspace-1>  c:;
ionMessages' '-cp' 'C:\Users\YZRD25\AppData\Roaming\Code\User\wo
Insertion Sort time = 6ms
Primitive Sort time = 1ms
Parallel Sort time = 2ms
PS C:\Users\YZRD25\Documents\GitHub\ai-masters-workspace-1>
```

Complexity Analysis

Insertion Sort best case: O(n)

Insertion Sort typical case: O(n^2)

Insertion Sort worst case: O(n^2)

Primitive (double-pivot) Sort Best Case: O(n logn)

Primitive (double-pivot) Sort typical case: O(n logn)

Primitive (double-pivot) Sort worst case: O(n^2)

Parallel Sort best case: O(n logn)

Parallel Sort typical case: O(n logn)

Parallel Sort worst case: O(n logn)

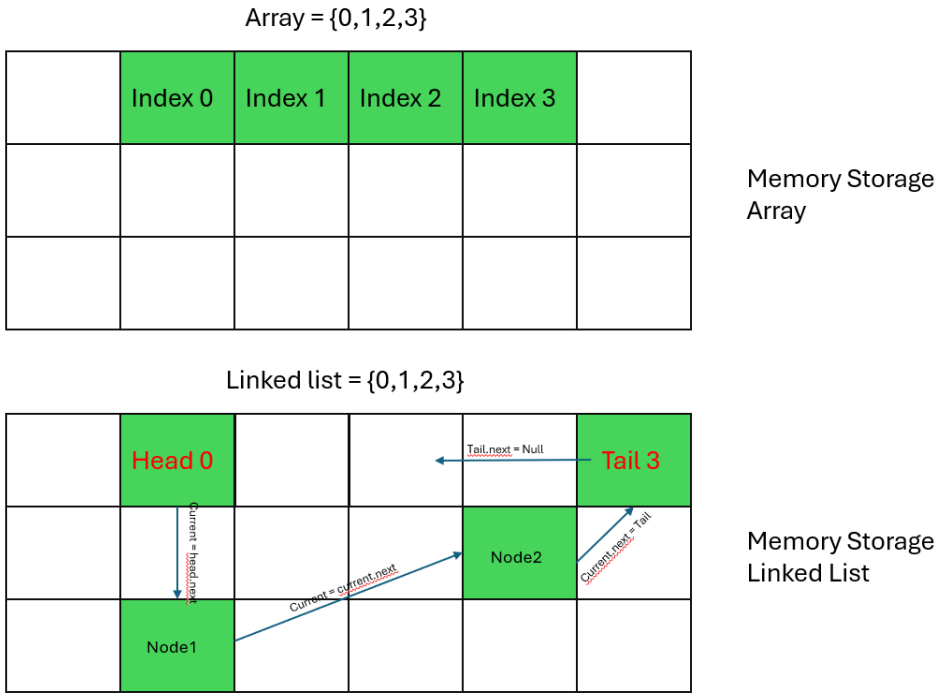
Question Three

25 Marks

Suppose a program builds and manipulates a linked list:

What two special nodes would the program typically keep track of? Describe two common uses for the null reference in the node of the linked lists.

The two special nodes a linked list program would like to keep track of are the Head and the Tail Nodes. Since a linked list is stored randomly in memory, unlink an array which is stored serially, there is no inherent index property to the structure. Therefore, when looping through or interrogating the linked list, you cannot use the index values or length of the structure as you would when using an array, instead you create pointers that move along the linked list. These pointers can utilize the "Head" to know which Node begins the linked list, and the "Tail" to know which Node ends the linked list.



Two common uses for the null reference are as follows:

1. The null reference can be used to show that a list is empty. If a function is called to return a value on a linked list that has no Nodes, the function will fail, hence if the list is empty return Null.
2. The Null reference can be used to show that a pointer has reached the end of the linked list as well. A pointer would know when it reaches the end of the list when Pointer.next == Null
3. One more reason why tracking the Null reference is useful is because it allows you to Remove and Insert Nodes effectively. When removing a Node, it's .next attribute needs to be set to Null, while when appending a Node to a Linked List you will need to set the .next Null reference of the tail to the New Node while the New Node's .next reference is set to Null.

Question Four

(25 Marks)

Consider the following Java Code:

What is the output when this code is executed?

```
import java.util.LinkedList;
import java.util.ListIterator;
class Main {
  public static void main(String[] args) {
LinkedList<String> myLList = new LinkedList<String>();
myLList.addFirst("Christophe");
myLList.addFirst("Stephen");
myLList.addLast("Christina");
ListIterator<String> myiterator = myLList.listIterator();
myiterator.next();
myiterator.next();
myiterator.add("Reem");
myiterator.previous();
myiterator.add("Ahmad");
myiterator.add("Khaled");
myiterator.previous();
myiterator.remove();
System.out.println(myLList);
myiterator.next();
myiterator.next();
    System.out.println("MyList Backward iteration :");
      while(myiterator.hasPrevious())
      System.out.println(myiterator.previous());
      System.out.println("---------------------");
  }
      }
```

Expected Output

The expected output of the function is as follows:

[Stephan, Christophe, Ahmad, Reem, Christina]

MyList Backward iteration :

Christina

Reem

Ahmad

Christophe

Stephan

Justification

```
import java.util.LinkedList; //import Linked List Module

import java.util.ListIterator; // Import ListIterator Module

class Main {

  public static void main(String[] args) {

//create LinkedList containing String Values called myLList

LinkedList<String> myLList = new LinkedList<String>();


//Add value "Christophe" at Head [Christophe]

myLList.addFirst("Christophe");


//Add value "Stephen" at Head [Stephen, Christophe]

myLList.addFirst("Stephen");
```

```
//Add value "Christina" at Tail [Stephen, Christophe, Christina]
myLList.addLast("Christina");

//Create an iterator on myLList
ListIterator<String> myiterator = myLList.listIterator();

//moves iterator between Stephan and Christophe
myiterator.next();

//moves iterator between Christophe and Christina
myiterator.next();

//adds value "Reem" at current index [Stephan, Christophe, Reem, Christina]
//moves iterator between Reem and Christina
myiterator.add("Reem");

//moves iterator back one between Christophe and Reem
myiterator.previous();

//adds value "Ahmad" at current index [Stephan, Christophe, Ahmad, Reem, Christina]
//moves iterator between Ahmad and Reem
myiterator.add("Ahmad");
```

```
//Adds value "Khaled" at current index [Stephan, Christophe, Ahmad, Khaled, Reem, Christina]

//moves iterator between Khaled and Reem

myiterator.add("Khaled");


//Moves iterator between Ahmad and Khaled

myiterator.previous();


//removes Node at current index "Khaled" [Stephan, Christophe, Ahmad, Reem, Christina]

myiterator.remove();


// prints list in current state Ouput: [Stephan, Christophe, Ahmad, Reem, Christina]

System.out.println(myLList);


//move iterator between Reem and Christina

myiterator.next();


//move iterator between Christina and Null

myiterator.next();


  //prints message "MyList Backward iteration :"

  System.out.println("MyList Backward iteration :");
```

```
    //While the iterator's previous value is not Null (Ends at Head)
    while(myiterator.hasPrevious())

    //print The Node
    System.out.println(myiterator.previous());

    //print message : "---------------------"
    System.out.println("---------------------");
  }
}
```

Assessment

1. As indicated on the syllabus, the project proposal is 30% of your grade.
2. You will be graded on the quality your project: originality, technical soundness, and report quality (grammatical errors, clarity, etc.).